

Lecture 3:

SQL1, Basic SQL

Instructor: Nikos Tziavelis

Reference:

*A First Course in Database Systems,
3rd edition, Chapter 2.3, 6.1*

Slides credit:
Sheldon Finkelstein

Summary of Previous Lecture

- Data model
- Relation Schema
 - Attributes or column names
 - Tuple or row
 - Schema
- Relation Instance
- Relational Database Schema
- A Relational Database Instance (“a database”)
 - An Instance of a Relational Database Schema
- Logical and Physical Data Independence

Recap: Instance of a Database Schema

- An *instance of a database schema* $\{R_1, \dots, R_k\}$ (or a *database instance* in short) is a set $\{r_1, \dots, r_k\}$ of relations such that r_i is an instance of R_i , for $1 \leq i \leq k$.

Student	studentID	name	major	gender	avgGPA
	112	Ann	Computer Science	F	3.95
	327	Bob	Computer Science	M	3.90
	835	Carl	Physics	M	4.00

Course	courseID	description	department	Teach, Enroll, Professor, ...
	CMPS101	Algorithms	CS	
	BINF223	Intro. to Bio.	Biology	

Recap: What is a Data Model?

- A *data model* is a mathematical formalism that consists of three parts:
 1. Structure of the data
 2. Operations on the data
 3. Constraints on the data
- This course focuses mainly on the relational data model

Two Query Languages

- Codd proposed two different query languages for the relational data model.
 - *Relational Algebra*
 - Queries are expressed as a sequence of operations on relations
 - Procedural language
 - *Relational Calculus*
 - Queries are expressed as formulas of first-order logic
 - Declarative language
- **Codd's Theorem:** The Relational Algebra query language has the same *expressive power* as the Relational Calculus query language.

Procedural vs. Declarative Languages

- **Procedural program**
 - The program is specified as a sequence of operations to obtain the desired the outcome. I.e., *how* the outcome is to be obtained.
 - E.g., Java, C, ...
- **Declarative program**
 - The program specifies *what* is the expected outcome, and not *how* the outcome is to be obtained.
 - E.g., Scheme, Ocaml, ...

An Example: Travel from Baskin Engineering, UC Santa Cruz to Soda Hall, UC Berkeley

Declarative (non-procedural):

- Go from Baskin Engineering, at McLaughlin Dr. and Heller Dr. in Santa Cruz, CA, to Soda Hall, at Hearst Ave. and Oxford St. in Berkeley, CA

An Example: Travel from Baskin Engineering, UC Santa Cruz to Soda Hall, UC Berkeley

Procedural (first part)

- Turn left onto Heller Dr,
- Turn left onto Empire Grade
- Continue onto High St
- High St turns right and becomes Storey St,
- Turn left onto King St
- Turn left onto CA-1 S/Mission St
- Continue to follow CA-1 S
- Take the CA-17 exit on the left toward San Jose/Oakland
- Continue onto CA-17 N
- Keep left to continue on I-880 N

An Example: Travel from Baskin Engineering, UC Santa Cruz to Soda Hall, UC Berkeley

Procedural (continued):

- Keep right at the fork, follow signs for CA-24/I-980/Walnut Cr.
Continue onto I-980 E
- Keep left to continue on CA-24 E
- Take the exit toward 51st Street
- Turn right onto 52nd St
- Take the 1st left onto Shattuck Ave
- Turn right onto University Ave
- Turn left onto Oxford St
- Turn right onto Hearst Ave
- Turn left; destination will be on the right

SQL – Structured Query Language

- Is SQL a procedural or a declarative language?
 - SQL is usually described as declarative, but it's not fully declarative.
- SQL is the principal language used to describe and manipulate data stored in relational database systems.
 - Frequently pronounced as “Sequel”, but formally it's “Ess Cue El”
 - Not exactly the same as Codd's relational algebra or relational calculus
- Several iterations of the standard from cooperating groups
 - SQL-86, SQL-89, **SQL-92 (SQL2)**, SQL:1999 (SQL3), SQL:2003, SQL:2006, SQL:2008, SQL:2011
 - ANSI (American National Standards Institute)
 - ISO (International Organization for Standards)
 - Implementations usually offer their own extensions of SQL.

SQL : 2 sublanguages

- *Data Definition Language (DDL)*
 - Sublanguage of SQL used to create, delete, modify the definition of tables and views.
 - For declaring database schemas.
- *Data Manipulation Language (DML)*
 - Sublanguage of SQL that allows users to insert, delete, and modify rows of tables and pose queries to retrieve data.
 - For asking questions about the database and modifying the database.

SQL : 2 sublanguages

- ***Data Definition Language (DDL)***
 - Sublanguage of SQL used to create, delete, modify the definition of tables and views.
 - For declaring database schemas.
- ***Data Manipulation Language (DML)***
 - Sublanguage of SQL that allows users to insert, delete, and modify rows of tables and pose queries to retrieve data.
 - For asking questions about the database and modifying the database.

Relations in SQL

Two types of Relations:

1. Stored relations (or Tables)

- These are tables that contain tuples and can be modified or queried.

2. Views

- Views are relations that are defined in terms of other relations, but they are not stored.
- They are logical tables, which can be used in queries as if they were tables.

Data Types in SQL

- INT or INTEGER
 - Analogous to int in C
- SMALLINT
 - Analogous to short int in C
- FLOAT(p), FLOAT and REAL (Implementation-specific)
- DOUBLE PRECISION
 - Analogous to `double` in C
- DECIMAL(n,d), NUMERIC(n,d) – almost the same meaning.
 - Total of n decimal digits; d of them to the right of the decimal point
- BOOLEAN or BOOL: TRUE, FALSE, UNKNOWN

Data Types in SQL

- CHAR(n): fixed-length string of up to n characters (which is blank-padded on the right to be exactly n characters)
- VARCHAR(n): string of up to n characters, not blank-padded
- BIT(n)
- BIT VARYING(n):
- DATE, TIME, TIMESTAMP, INTERVAL
 - Separate data types
 - Constants are character strings of a specific form, e.g., DATE '2017-09-13' and TIME '16:45:33'
 - Consult documentation for operation results
 - E.g., Subtracting one TIME from another results in an INTERVAL

NULL is a special value in SQL used to represent unknown values for any data type.

SQL: Data Definition Language (DDL)

Defining a Table

movieTitle	movieYear	length	genre	studioName	producerC#
------------	-----------	--------	-------	------------	------------

Movies

```
CREATE TABLE Movies (  
    movieTitle      CHAR(100),  
    movieYear       INT,  
    length          INT,  
    genre           CHAR(10),  
    studioName      CHAR(30),  
    producerC#      INT  
);
```

Think of producerC# as the Certificate Number for the movie's producer, where Certificate Number is a key uniquely identifying a Movie Executive.

SQL: Data Definition Language (DDL)

Defining a Table

starName	address	gender	birthdate
----------	---------	--------	-----------

MovieStar

```
CREATE TABLE MovieStar (  
    starName      CHAR(30),  
    address       VARCHAR(255),  
    gender        CHAR(1),  
    birthdate     DATE  
);
```

SQL: Data Definition Language (DDL)

Deleting a Table

movieTitle	movieYear	length	genre	studioName	producerC#
------------	-----------	--------	-------	------------	------------

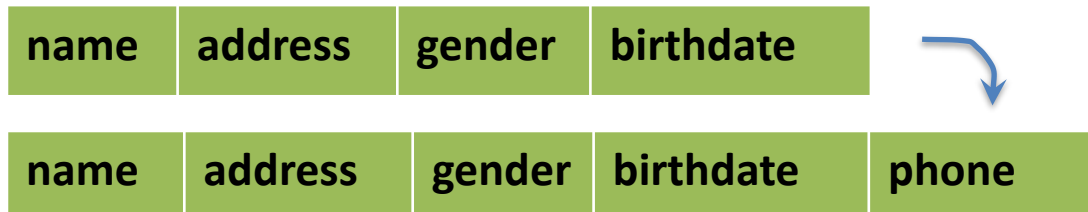
Movies

DROP TABLE Movies;

- The entire table is removed from the database schema.

SQL: Data Definition Language (DDL)

Modifying a Table



```
ALTER TABLE MovieStar ADD phone CHAR(16);
```

Adds an attribute “phone” with type CHAR(16) to the table MovieStar.

name	address	gender	phone
------	---------	--------	-------

```
ALTER TABLE MovieStar DROP birthdate;
```

SQL: Data Definition Language (DDL)

Default Values

- If a row does not have a value specified for an attribute, the default value is NULL
 - e.g., Adding a new column to existing table
- A specific default value can be set using the **DEFAULT** keyword

```
CREATE TABLE MovieStar (  
    starName      CHAR(30),  
    address       VARCHAR(255) DEFAULT 'Hollywood',  
    gender        CHAR(1),  
    birthdate     DATE  
);
```

More Examples of Default Values

```
CREATE TABLE MovieStar (  
    starName      CHAR(30),  
    address       VARCHAR(255)  DEFAULT    'Hollywood',  
    gender        CHAR(1)       DEFAULT    'U',  
    birthdate     DATE          DEFAULT    DATE '1990-08-26'  
);
```

```
ALTER TABLE MovieStar ADD phone CHAR(16) DEFAULT 'unlisted';
```

Recap: Superkey & Key

A *superkey* is a subset S of the attributes such that:

No two tuples have the same value for **ALL** attributes in S

A *key* is a *superkey* that is minimal

- No proper subset can be a *superkey*

Flightinfo (*flightnumber*, *seatnumber*, *firstname*, *lastname*)

Are the followings key?

- {flightnumber}
- {seatnumber}
- {flightnumber, seatnumber}
- {flightnumber, seatnumber, lastname}

Flightinfo (*flightnumber*, *seatnumber*, *firstname*, *lastname*)

Recap: Superkey & Key

A relation may have multiple keys

Student(studentid, name, address, gpa)

Student(studentid, name, address, gpa)

Student(studentid, name, address, gpa)

Declaring Keys

Two ways to declare a single attribute to be a key in the CREATE TABLE statement:

```
CREATE TABLE MovieStar (  
    starName    CHAR(30) PRIMARY KEY,  
    address     VARCHAR(255),  
    gender      CHAR(1),  
    birthdate   DATE  
);
```

```
CREATE TABLE MovieStar (  
    starname    CHAR(30),  
    address     VARCHAR(255),  
    gender      CHAR(1),  
    birthdate   DATE,  
    PRIMARY KEY (starName)  
);
```


Declaring Keys (continued)

If the key consists of multiple attributes, then those attributes can be declared as a key as follows:

```
CREATE TABLE Movies (  
    movieTitle    CHAR(100),  
    movieYear     INT,  
    length        INT,  
    genre         BOOLEAN,  
    studioName    CHAR(30),  
    producerC#    INT,  
    PRIMARY KEY (movieTitle, movieYear)  
);
```

Two types of *keys* in SQL

1. PRIMARY KEY
2. UNIQUE

PRIMARY KEY	UNIQUE
Does not allow NULL	Allows NULL
At most 1 in a TABLE	Can be multiple in a TABLE

PRIMARY KEY vs. UNIQUE

```
CREATE TABLE MovieStar (  
    starName    CHAR(30),  
    address     VARCHAR(255),  
    gender      CHAR(1),  
    birthdate   DATE,  
    PRIMARY KEY (starName)  
);
```

- None of the rows in MovieStar can have NULL *starName* values.
- Rows are uniquely identified by their *starName* values.
- There can be at most one Primary Key for a table.

```
CREATE TABLE MovieStar (  
    starName    CHAR(30),  
    address     VARCHAR(255),  
    gender      CHAR(1),  
    birthdate   DATE,  
    UNIQUE (starName)  
);
```

- Rows in MovieStar can contain NULL *starName* values.
- Rows in MovieStar with non-NULL *starName* values are uniquely identified by their *name* values.
- There can be multiple unique constraints for a table, in addition to a primary key.

More on NULL

```
CREATE TABLE MovieStar (  
    starName      CHAR(30)  PRIMARY KEY,  
    address       VARCHAR(255) NOT NULL DEFAULT 'Hollywood',  
    gender        CHAR(1),  
    birthdate     DATE NOT NULL  
);
```

- If no value is entered for an attribute, and no DEFAULT value is specified for that attribute, and **that attribute can be NULL, then the value of the attribute will be NULL.**
 - **NOT NULL** constraint prevents a column from having NULL values.
 - Attributes that don't have NOT NULL specified may be NULL.
 - ... but remember that attributes in the PRIMARY KEY can't be NULL.

Foreign Key

- A constraint (NOT a key)
- A database with 5 tables:
 - Movies(movieTitle, movieYear, length, genre, studioName, producerC#)
 - StarsIn(movieTitle, movieYear, starName)
 - MovieStar(starName, address, gender, birthdate)
 - MovieExec(execName, address, cert#, netWorth)
 - Studio(studioName, address, presC#)

Requirement: any *presC#* in *Studio* must also exist as *cert#* in *Movies*

Syntax: **FOREIGN KEY (presC#) REFERENCES MovieExec(cert#)**

Database Schema for Our Running Example

- Let's assume we have a database schema with five relation schemas.

Movies(movieTitle, movieYear, length, genre, studioName, producerC#)

StarsIn(movieTitle, movieYear, starName)

MovieStar(starName, address, gender, birthdate)

MovieExec(execName, address, cert#, netWorth)

Studio(studioName, address, presC#)

*In this schema, **cert#** is just an attribute for a MovieExec's “certificate number”; producerC# and presC# should refer to the cert# of the producer of a Movie and president of a Studio, respectively. Nothing special about these column names/attributes for SQL—it's just an example.*

*And note that the # symbol is **not** allowed in identifiers (such as column names and table names) in PostgreSQL, even though our textbook uses it.*

CREATE TABLE for MovieExec

```
CREATE TABLE MovieExec (  
    execName      CHAR(30),  
    address       VARCHAR(255),  
    cert#         INT PRIMARY KEY,  
    netWorth      INTEGER  
);
```

MovieExec(execName, address, cert#, netWorth)

CREATE TABLE for Studio

```
CREATE TABLE Studio (  
    studioName      CHAR(30) PRIMARY KEY,  
    address         VARCHAR(255),  
    presC#          INT,  
    FOREIGN KEY (presC#) REFERENCES MovieExec(cert#)  
);
```

Studio(studioName, address, presC#)

- Every presC# in Studio must be the Primary Key (cert#) of a movie exec in the MovieExec table.
 - But for each movie exec in the MovieExec table, there can be n studios in the Studios table for which that movie exec is president, where n can be 0, 1, 2, ..., 17, ...

CREATE TABLE for Movies

```
CREATE TABLE Movies (  
    movieTitle    CHAR(100),  
    movieYear     INT,  
    length        INT,  
    genre         BOOLEAN,  
    studioName    CHAR(30),  
    producerC#    INT,  
    PRIMARY KEY (movieTitle, movieYear),  
    FOREIGN KEY (studioName) REFERENCES Studio,  
    FOREIGN KEY (producerC#) REFERENCES MovieExec(cert#)  
);
```

Didn't have to say:
FOREIGN KEY (studioName)
REFERENCES Studio (**studioName**),
because referencing attributes and
referenced attributes are the same.

Movies(movieTitle, movieYear, length, genre, studioName, producerC#)

- Every **studioName** in Movies must be the Primary Key (**studioName**) of a studio in the Studio table.
 - But for each **studioName** in the Studio table, there can be **n** movies in the Movies table which have that **studioName**, where **n** can be 0, 1, 2, ..., 17, ...
- Every **producerC#** in Movies must be the Primary Key (**cert#**) of a movie exec in the MovieExec table.
 - But for each movie exec in the MovieExec table, there can be **n** movies in the Movies table for which that movie exec is producer, where **n** can be 0, 1, 2, ..., 17, ...

CREATE TABLE for MovieStar

```
CREATE TABLE MovieStar (  
    starName      CHAR(30) PRIMARY KEY,  
    address       VARCHAR(255),  
    gender        CHAR(1),  
    birthdate     DATE  
);
```

MovieStar(starName, address, gender, birthdate)

CREATE TABLE for StarsIn

```
CREATE TABLE StarsIn (  
    movieTitle CHAR(100),  
    movieYear INT,  
    starName CHAR(30),  
    PRIMARY KEY (movieTitle, movieYear, starName),  
    FOREIGN KEY (movieTitle, movieYear) REFERENCES Movies,  
    FOREIGN KEY (starName) REFERENCES MovieStar  
);
```

StarsIn(movieTitle, movieYear, starName)

- Every (movieTitle, movieYear) in StarsIn must be the Primary Key of a movie in the Movies table.
 - But for each (movieTitle, movieYear) in the Movies table, there can be n Movies in the StarsIn table which have that (movieTitle, movieYear) , where n can be 0, 1, 2, ..., 17, ...
- Every starName in StarsIn must be the Primary Key of a movie star in the MovieStars table.
 - But for each starName in the MovieStar table, there can be n Movies in the StarsIn table which have that starName, where n can be 0, 1, 2, ..., 17, ...

PostgreSQL Meta Commands

- \l
 - List the databases.

After you connect to a database, you can perform the following:

- \dt
 - List the tables of a database.
- \d table_name
 - List the columns for a specific table_name.

Tables are not necessarily listed in the order in which you created them. That's okay!

Identifiers, such as table names and column names will appear in lowercase, not in the case you used when you created them. That's okay!

You can use the “double quote” symbol (”) around an identifier if you really care about case ... but then you'll always have to use double quotes around that identifier.

Example: “movieYear” versus movieYear

<https://www.geeksforgeeks.org/postgresql-psql-commands/>

SQL : 2 sublanguages

- *Data Definition Language (DDL)*
 - Sublanguage of SQL used to create, delete, modify the definition of tables and views.
 - For declaring database schemas.
- ***Data Manipulation Language (DML)***
 - Sublanguage of SQL that allows users to insert, delete, and modify rows of tables and pose queries to retrieve data.
 - For asking questions about the database and modifying the database.

A Simple SQL Query

- Find all movies produced by Disney Studios in 1990.

SELECT *

FROM Movies

WHERE studioName = 'Disney' AND movieYear = 1990;

Basic Form of a SQL Query

- Basic form:

SELECT [**DISTINCT**] $c_1, c_2, \dots, c_m$
FROM $R_1, R_2, \dots, R_n$
[**WHERE** *condition*]

Attribute names

Relation names

- We will focus on one relation R_1 for now.
- What is the semantics (that is, the meaning) of this query?

A Simple SQL Query

- Find all movies produced by Disney Studios in 1990.

SELECT *

FROM Movies

WHERE studioName = 'Disney' AND movieYear = 1990;

The symbol “*” is a shorthand for all attributes of relations in the FROM clause.

Movies

movieTitle	movieYear	length	genre	studioName	producerC#
Pretty Woman	1990	119	true	Disney	999
Monster's Inc.	1990	121	true	Dreamworks	223
Jurassic Park	1998	145	NULL	Disney	675

A Simple SQL Query (Result)

- Find all movies produced by Disney Studios in 1990.
SELECT *
FROM Movies
WHERE studioName = 'Disney' AND movieYear = 1990;

Result

movieTitle	movieYear	length	genre	studioName	producerC#
Pretty Woman	1990	119	true	Disney	999

An Even Simpler SQL Query

- Find all movies.

```
SELECT *  
FROM Movies;
```

Equivalent to:

```
SELECT *  
FROM Movies  
WHERE TRUE;
```

Movies	movieTitle	movieYear	length	genre	studioName	producerC#
	Pretty Woman	1990	119	true	Disney	999
	Monster's Inc.	1990	121	true	Dreamworks	223
	Jurassic Park	1998	145	NULL	Disney	675

An Even Simpler SQL Query (Result)

- Find all movies.

```
SELECT *
```

```
FROM Movies;
```

Result

movieTitle	movieYear	length	genre	studioName	producerC#
Pretty Woman	1990	119	true	Disney	999
Monster's Inc.	1990	121	true	Dreamworks	223
Jurassic Park	1998	145	NULL	Disney	675

A Simple SQL Query with Projection

- Return the title and year of all movies.

```
SELECT movieTitle, movieYear  
FROM Movies;
```

A Projection Query:

- Only a subset of attributes from the relation(s) in the FROM clause is selected.

Result

movieTitle	movieYear
Pretty Woman	1990
Monster's Inc.	1990
Jurassic Park	1998

A Simple SQL Query with Projection and Selection

- Return the title and year of all movies produced by Disney Studios in 1990.

```
SELECT movieTitle, movieYear
```

```
FROM Movies
```

```
WHERE studioName = 'Disney' AND movieYear = 1990;
```

Result

movieTitle	movieYear
Pretty Woman	1990

Distinct: Sets vs Multisets/Bags

- Return the years of all movies with length less than 140.

```
SELECT movieYear  
FROM Movies  
WHERE length < 140;
```

Result

movieYear
1990
1990

Multiset or bag semantics

```
SELECT DISTINCT movieYear  
FROM Movies  
WHERE length < 140;
```

Result

movieYear
1990

Set semantics

Bags (or Multisets) vs. Sets

- From basic set theory:
- Every element in a set is distinct
 - E.g., $\{2,4,6\}$ is a set but $\{2,4,6,2,2\}$ is not a set
 - ... or is same set as $\{2,4,6\}$
- A bag (or multiset) may contain repeated elements.
 - E.g., $\{\{2,4,6\}\}$ is a bag. So is $\{\{2,4,6,2,2\}\}$.
 - Note that double set brackets in $\{\{2,4,6\}\}$ indicate it's a bag, not a set
 - Equivalently written as $\{2[3],4[1],6[1]\}$.
- The order among elements in a set or bag is not important
 - E.g., $\{2,4,6\} = \{4,2,6\} = \{6,4,2\}$
 - $\{\{2,4,6,2,2\}\} = \{\{2,2,2,6,4\}\} = \{\{6,2,2,4,2\}\}$.

Aliasing Attributes

- Allows you to rename the attributes of the result.
- Example: Return the title and length of all movies as attributes name and duration.

```
SELECT movieTitle AS name, length AS duration  
FROM Movies;
```

Result

name	duration
Pretty Woman	119
Monster's Inc.	121
Jurassic Park	145

The keyword AS is optional when you alias in SELECT clause.

Expressions in the SELECT Clause

- Expressions are allowed in the SELECT clause.
- Return the title and length of all movies as name and duration in seconds (durationInSeconds).

```
SELECT movieTitle AS name,  
       length * 60 AS durationInSeconds  
FROM Movies;
```

Result

name	durationInSeconds
Pretty Woman	7140
Monster's Inc.	7260
Jurassic Park	8700

Constants in the Result

- Constants can also be in the SELECT clause.
- Every row will have the same constant specified in the SELECT clause.

```
SELECT movieTitle AS name, length * 60 AS  
       durationInSeconds, 'seconds' AS inSeconds  
FROM Movies;
```

Result

name	durationInSeconds	inSeconds
Pretty Woman	7140	seconds
Monster's Inc.	7260	seconds
Jurassic Park	8700	seconds

More on Conditions in the WHERE Clause

WHERE studioName = 'Disney' AND movieYear = 1990;

- Comparison operators:

- =, <>, <, >, <=, >=, LIKE

Can use != instead of <>

- Equal, not equal, less than, greater than, less than or equal, greater than or equal, like

- Example:

WHERE movieYear <= 1990

- Logical connectives:

- AND, OR, NOT

- Examples:

WHERE movieYear >= 1990 AND movieYear <= 2005

That can also be written as: WHERE movieYear BETWEEN 1990 AND 2005

WHERE NOT (studioName = 'DISNEY' AND movieYear <= 1990)

- Arithmetic expressions:

- +, -, *, /, etc.

- Example:

WHERE ((length/60) > 2 OR (length < 100)) AND movieYear > 2000

More on the Conditions in the WHERE Clause (cont'd)

- In general, the WHERE clause consists of a boolean combination of conditions using logical connectives AND, OR, and NOT.
- Each condition is of the form
expression op expression
 - *expression* is a column name, a constant, or an arithmetic or string expression
 - *op* is a comparison operator (=, <>, <, >, <=, >=, LIKE)



(More on this very soon)

String Comparisons

- Strings are compared in lexicographical order.
 - Let $a_1.a_2. \dots .a_n$ and $b_1.b_2. \dots b_m$ be two strings.
 - Then $a_1.a_2. \dots .a_n < b_1.b_2. \dots b_m$ if either:
 1. $a_1.a_2. \dots .a_n$ is a proper prefix of $b_1.b_2. \dots b_m$, or,
 2. For some $1 \leq i \leq n$, we have $a_1=b_1, a_2=b_2, \dots, a_{i-1}=b_{i-1}$ and $a_i < b_i$.
- Examples:
 - 'Pretty' < 'Pretty Woman',
 - 'butterfingers' < 'butterfly'

Pattern Matching in SQL

- LIKE operator
 - s LIKE p, s NOT LIKE p
 - s is a string, p is a pattern
- '%' (stands for 0 or more arbitrary chars)
- '_' (stands for exactly one arbitrary char)

```
SELECT *  
FROM Movies  
WHERE movieTitle LIKE 'Jurassic%'
```

```
SELECT *  
FROM Movies  
WHERE movieTitle LIKE 'Juras_ic%'
```

Pattern Matching in SQL

- How do you match titles with a quote symbol in them?
 - E.g., 'Monster's Inc.'

```
SELECT *  
FROM Movies  
WHERE movieTitle LIKE 'Monster's Inc.'
```

Wrong!!!

```
WHERE movieTitle LIKE 'Monster"s Inc.'
```

Pattern Matching in SQL

- How do you match just a quote symbol?

WHERE movieTitle LIKE ''' // matches ' (single quote)

- How do you match two quote symbols?

WHERE movieTitle LIKE '""' // matches " (two single quotes)

- How do you match % or _?

WHERE movieTitle LIKE '!%%!_' ESCAPE '!'
// ! could be any character

Would this match: 'ABC%D' '%ABC' '%ABC_' 'ABCD' '%_'
 No No Yes No Yes

- You can even match quote using an ESCAPE character.

WHERE movieTitle LIKE ':':" ESCAPE ':' // matches " (two single quotes)

Comparing Dates and Times

- Reminder about DATE, TIME, TIMESTAMP and INTERVAL
 - Separate data types
 - Constants are character strings of a specific form
 - DATE '2021-01-13' or DATE '01/13/21'
 - TIME '16:45:33'
 - TIMESTAMP '2021-01-13 16:45:33'
 - INTERVAL '2 HOURS 30 MINUTES'
 - See Chapter 6.1.5 of Textbook or [PostgreSQL manual](#)
- DATE, TIME, TIMESTAMP and INTERVAL types can be compared using ordinary comparison operators.
 - ... WHERE ReleaseDate <= DATE '2020-06-19' ...
- There are some implementation-specific differences on formats.
- Localization of types, e.g., for European dates.
 - Character sets can also be localized.
- In PostgreSQL, if myTimestamp is a TIMESTAMP value, then DATE(myTimestamp) is the DATE value for that TIMESTAMP.

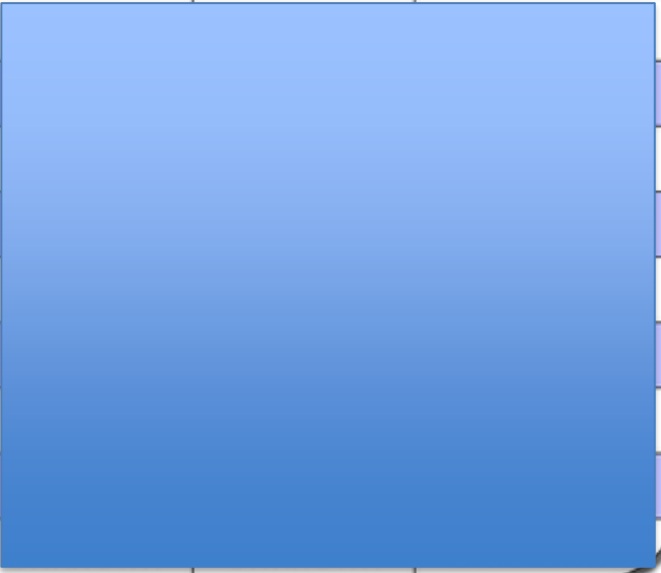
This could be helpful for Lab2. But it's not in the SQL standard.

SQL's Three-Valued Logic

Students	studentID	name	major	gender	avgGPA
	112	Ann	Computer Science	F	NULL
	327	Bob	NULL	M	3.90
	835	Carl	Physics	M	4.00

- What is the result of this comparison?
major = 'Computer Science' AND avgGPA > 3.00
- Three-valued logic TRUE, FALSE, UNKNOWN
- If major is NULL, then “major= 'Computer Science' ” evaluates to UNKNOWN.
- If avgGPA is NULL, then “avgGPA > 3.00 ” evaluates to UNKNOWN.

SQL's Three-Valued Logic (cont'd)

p	q	$p \text{ OR } q$	$p \text{ AND } q$	$p = q$
True	True			
True	False			
True	Unknown			
False	True			
False	False			
False	Unknown			
Unknown	True			
Unknown	False			
Unknown	Unknown			

p	NOT p
True	
False	
Unknown	

SQL's Three-Valued Logic: Truth Table

p	q	$p \text{ OR } q$	$p \text{ AND } q$	$p = q$
True	True	True	True	True
True	False	True	False	False
True	Unknown	True	Unknown	Unknown
False	True	True	False	False
False	False	False	False	True
False	Unknown	Unknown	False	Unknown
Unknown	True	True	Unknown	Unknown
Unknown	False	Unknown	False	Unknown
Unknown	Unknown	Unknown	Unknown	Unknown

p	NOT p
True	False
False	True
Unknown	Unknown

Example of Three-Value Logic

```
SELECT *  
FROM Students  
WHERE major = 'Computer Science' AND avgGPA > 3.00;
```

- If the condition evaluates to UNKNOWN, then the tuple will not be returned.
- Both Ann and Bob will not be returned.
- In fact, none of the tuples will be returned.
- Is UNKNOWN the same as FALSE?
 - No; why not?
 - What happens if the condition was the following?
NOT (major = 'Computer Science' AND avgGPA > 3.00)

Some Facts About NULL

- Almost all comparisons with NULL will evaluate to UNKNOWN. If salary is NULL, then the following will be UNKNOWN:
 - salary = 10
 - salary <> 10
 - 90 > salary OR 90 <= salary
 - salary = NULL
 - salary <> NULL
- Use of IS NULL and IS NOT NULL
 - salary IS NULL
 - will be TRUE if salary is NULL, FALSE otherwise
 - salary IS NOT NULL
 - will be TRUE if salary isn't NULL, FALSE otherwise

Some More Examples For NULL

- If salary1 has value NULL and salary2 has value NULL, then what will be the value for these predicates?
 - salary1 = salary2
 - salary1 <> salary2
 - salary1 IS NULL
 - salary2 IS NOT NULL
 - salary1 IS NULL OR salary2 IS NOT NULL
- Write a query that returns the names of students whose Major is either Computer Science or NULL.
- Write a query that returns the names of students whose Major isn't NULL and whose avgGPA is NULL.

SQL's Three-Valued Logic

Students	studentID	name	major	gender	avgGPA
	112	Ann	Computer Science	F	NULL
	327	Bob	NULL	M	3.90
	835	Carl	Physics	M	4.00

- What is the result of this comparison?
 - `major = 'Computer Science' AND avgGPA > 3.00`
- Three-valued logic TRUE, FALSE, UNKNOWN
- If major is NULL, then “`major = 'Computer Science'`” evaluates to UNKNOWN.
- If avgGPA is NULL, then “`avgGPA > 3.00`” evaluates to UNKNOWN.

NULL, UNKNOWN and Empty Set

These are very different!

- An **Attribute** can have value **NULL**
- The result of a **Condition** can be **UNKNOWN**
 - SQL's Three-Valued Logic:
Not just TRUE and FALSE, but also UNKNOWN.
- The **Result of a Query** can be the **Empty Set**, meaning that there are no tuples in the result.
 - A table may also have no tuples in it.

Ordering the Result

```
SELECT [DISTINCT] <list of attributes>  
FROM   R1, R2, ..., Rn  
[WHERE condition]  
[ORDER BY <list of attributes>]
```

- ORDER BY presents the result in a sorted order.
- By default, the result will be ordered in ascending order, ASC.
 - For descending order on an attribute, you write DESC next to that attribute in the list of attributes.
 - Example appears on next slide.

ORDER BY

```
SELECT *  
FROM Movies  
WHERE studioName = 'Disney' AND movieYear = 1990  
ORDER BY length, movieTitle;
```

- The result will list movies that satisfy the condition in the WHERE clause in ascending (ASC) order of length, then by ascending title.
 - Shortest length movies will be listed first.
 - Among all movies with the same length, the movies will be sorted in ascending order of movieTitle.
- ORDER BY length DESC, movieTitle;
 - Longest length movies will be listed first.
 - Among all movies with the same length, the movies will be sorted in ascending order of movieTitle.
- What about: **ORDER BY length, movieTitle DESC;**

More on ORDER BY

- You can ORDER BY expressions, not just attributes
 - ORDER BY Quantity * Price
 - ORDER BY Quantity * Price DESC
- ORDER BY works with attributes that can have NULL values
 - NULL will probably be smallest or largest value.
 - Not specified by SQL standard, so it depends on the implementation.
 - For example, in PostgreSQL, DB2 and Oracle, NULL is the largest value, but in SQL Server and SAP HANA, NULL is the smallest value.
 - For purposes of questions on ORDER BY in a Gradiance assignment, please treat NULL as the smallest possible value, not the largest possible value.
 - If an exam requires ORDER BY with NULL, you'll be told whether NULL should be treated as the smallest or the largest value.

Meaning of an SQL Query with One Relation in the FROM Clause

```
SELECT [DISTINCT]  $c_1, c_2, \dots, c_m$   
FROM  $R_1$   
[WHERE condition]  
[ORDER BY <list of attributes [ASC | DESC] >]
```

- Let Result denote an empty multiset of tuples.
- For every tuple t from R_1 ,
 - if t satisfies *condition* (i.e., if condition evaluates to true), then add the tuple that consists of $c_1, c_2, \dots, c_m$ components of t into Result.
- If DISTINCT is in the SELECT clause, then remove duplicates in Result.
- If ORDER BY <list of attributes> exists, then order the tuples in Result according to ORDER BY clause.
- Return Result.

Practice Homework 2

- Define the following two relational schemas in SQL using reasonable datatypes for each attribute.
 - Sells (bar, beer, price): indicates the price of each beer sold at each bar. (Note that each bar can sell many beers and many bars can sell the same beer, at possibly different prices.)
 - Frequents (drinker, bar): indicates which drinkers frequent which bars. (Note that each drinker may frequent many bars and many drinkers may frequent the same bar.)
 - Likes (drinker, beer): indicates which drinker likes which beers. (Note that a drinker may like many beers, and many drinkers may like the same beer.)
- Add a column “size” to Sells relation schema using the “ALTER TABLE” command.
- Write an SQL query to retrieve all beers sold by the bar “99 bottles”.
- Write an SQL query to retrieve all beers that are priced above \$3.